Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación (FCC)
Carrera de Ingeniería de Software

Aplicaciones Distribuidas — TA-IND-01

Análisis crítico de Apache Cassandra

Replicación, consistencia y disponibilidad de lectura

Kevin Leonel García Bazurto
Paralelo A
13 de noviembre de 2026

https://github.com/Yukisan-uwo/TA-IND-01-Informe-Academico-Individual-Analisis-Critico-de-Sistemas-Distribuidos-Grupo

Resumen

Este informe analiza cómo Apache Cassandra distribuye los datos y mantiene disponible una consulta cuando dejan de responder algunos nodos. El análisis describe el papel del coordinador, la replicación por centro de datos y los niveles de consistencia. Se realizó una prueba con Cassandra 5.0.9 en Docker, tres nodos y un factor de replicación de tres. Después de insertar un registro con ALL, se repitió la misma consulta con ONE y QUORUM mientras se detenían dos nodos de forma sucesiva. Con los tres nodos activos, ambos niveles devolvieron el registro. Con una única réplica disponible, ONE completó la lectura y QUORUM devolvió un error de disponibilidad porque requería dos respuestas. La comparación documental con CockroachDB examina la distribución de datos, la coordinación de lecturas, la replicación, los fallos y la recuperación. El resultado del laboratorio muestra que la disponibilidad depende del nivel de consistencia seleccionado y del estado de las réplicas de cada dato. La prueba se limita a lecturas de un registro previamente escrito y permite identificar qué condiciones habría que estudiar para evaluar escrituras, latencia o particiones de red.

1. Introducción

Cassandra aborda el almacenamiento de grandes volúmenes de datos en varias máquinas. Divide el conjunto de datos para repartir la carga y conserva varias copias para que un nodo fuera de servicio no implique necesariamente la pérdida de acceso. Discord utilizó Cassandra para su historial de mensajes. Su equipo de ingeniería informó que a comienzos de 2022 su clúster tenía 177 nodos y almacenaba billones de mensajes [1]. Ese caso muestra tanto la escala alcanzada como las dificultades operativas de sostenerla.

Elegí Cassandra porque permite variar el nivel de consistencia de una operación sin cambiar el registro ni la consulta. Esta propiedad hace posible observar en un laboratorio pequeño una decisión que también importa en producción. Los estudios de Ferreira et al. y Redžepagić et al. examinan cómo la configuración de consistencia y replicación afecta el rendimiento en otros entornos [2], [3]. Sus mediciones aportan contexto para interpretar una práctica centrada en disponibilidad.

2. Arquitectura del sistema

La clave de partición se transforma en un token que permite localizar los nodos responsables del dato. En la tabla del laboratorio, `id` cumple esa función. La estrategia `NetworkTopologyStrategy` establece el número de réplicas por centro de datos. En `dc1`, el factor de tres coloca la partición del registro de prueba en los tres nodos disponibles [4].

El cliente `cqlsh` envía la consulta CQL a `taind-cas1`. Este nodo actúa como coordinador de la petición, localiza las réplicas y reúne las respuestas que exige el nivel seleccionado. También posee una réplica del registro. El papel de coordinador depende del nodo que recibe la petición. Los nodos intercambian información de pertenencia y actividad mediante *gossip* [4].En el bosquejo del Anexo A representé la consulta, el coordinador y las réplicas. En la Figura 1 añadí la terminal de PowerShell en Windows, la red interna de Docker y los tres contenedores. También indiqué que, en esta práctica, ejecuté `cqlsh` dentro del contenedor del primer nodo.

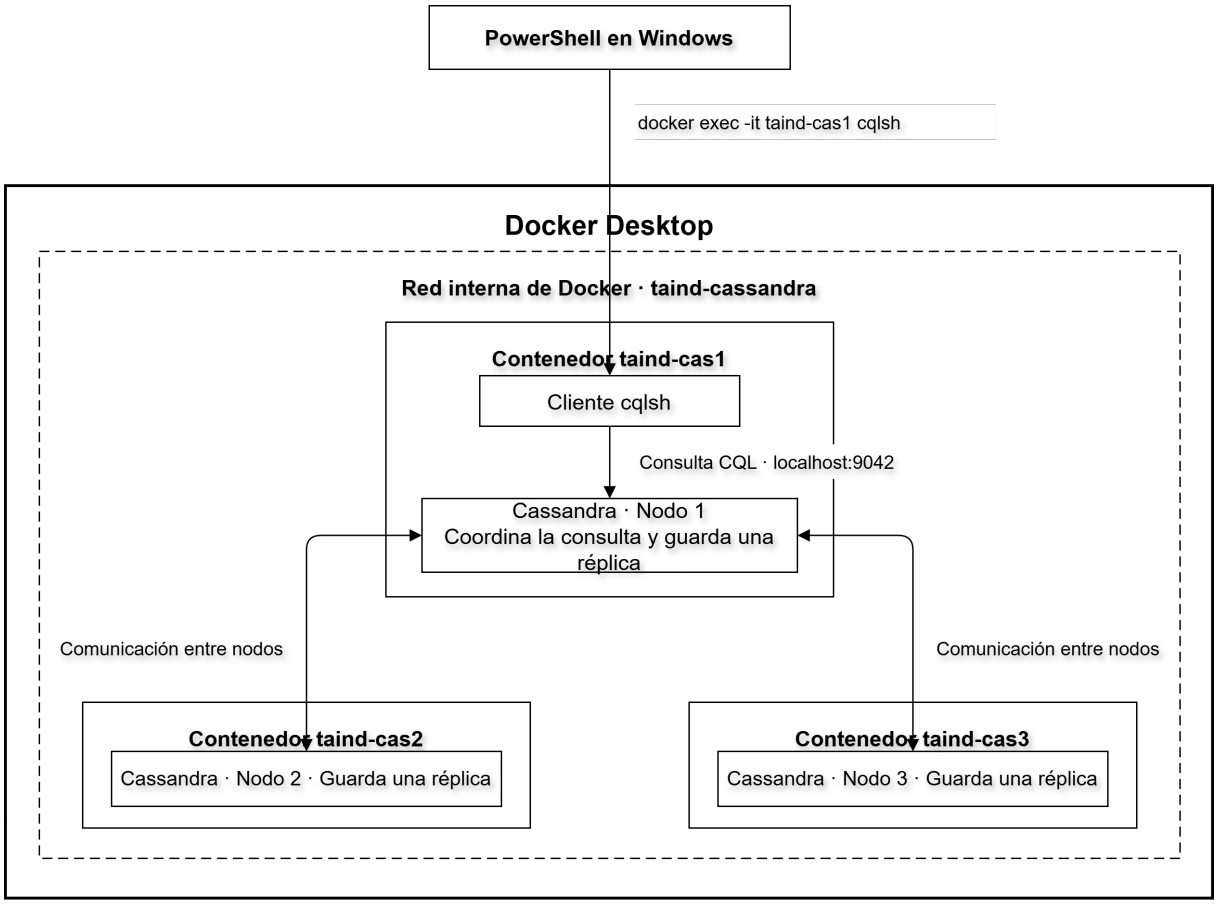


Figura 1: Arquitectura del clúster Cassandra utilizado en el experimento de lectura.

3. Análisis con los conceptos de la Unidad 1

3.1. Transparencias de distribución

Cuadro 1: Transparencias en el acceso a los datos de Cassandra.

Transparencia	Aplicación al sistema
Acceso	CQL ofrece una interfaz común para consultar datos desde un nodo del clúster.
Ubicación	El cliente identifica la fila por su clave. El coordinador resuelve qué nodos guardan la partición.
Migración	El movimiento de rangos entre nodos conserva la clave y la consulta del cliente. El operador sí participa en los cambios de topología.
Reubicación	Es parcial. El clúster puede cambiar la asignación de rangos mientras presta servicio. Una sesión conectada a un nodo que se apaga puede requerir reconexión.
Replicación	La consulta recupera una sola fila lógica. El factor de replicación y el nivel de consistencia son decisiones visibles de configuración.
Concurrencia	El motor resuelve conflictos de escrituras ordinarias mediante marcas de tiempo. Una condición que debe evaluarse atómicamente requiere una operación condicional.
Fallos	Es parcial. La caída de una réplica puede pasar inadvertida en una lectura que todavía alcanza su umbral. Si faltan respuestas, el cliente recibe un error.

El laboratorio muestra la transparencia parcial de fallos. La consulta conservó su sintaxis cuando se apagaron los contenedores. El fallo solo se hizo visible para `QUORUM` al quedar una réplica activa. La distribución y resolución de versiones siguen las reglas del modelo descrito por Cassandra [4].

3.2. CAP y PACELC

Ante una partición de red, CAP plantea un compromiso entre garantizar la visión más reciente de los datos y continuar atendiendo peticiones en cada lado de la separación. En Cassandra el resultado depende de la operación, su nivel de consistencia y las réplicas alcanzables. Con factor de tres, un coordinador que alcanza una sola réplica puede servir una lectura `ONE`, con el riesgo de que esa copia esté atrasada. Una lectura `QUORUM` requiere dos respuestas. Usar cuórum tanto en la escritura como en la lectura garantiza que ambos conjuntos compartan al menos una réplica, un mecanismo que mejora la visibilidad de escrituras confirmadas [2], [4].

PACELC extiende la discusión a la operación sin particiones. Esperar una respuesta permite completar la lectura con menos coordinación. Esperar una mayoría puede añadir demora si alguna réplica responde lentamente o está lejos. La magnitud de esa diferencia depende de la carga y la configuración [2], [3]. Mi prueba detuvo contenedores y observó si la lectura se completaba. Una prueba de partición tendría que aislar enlaces mientras los nodos siguen ejecutándose. Para estudiar la latencia harían falta consultas repetidas y mediciones comparables.

3.3. Modelos de comunicación

Entre `cqlsh` y Cassandra hay una relación cliente-servidor. El laboratorio usó el protocolo nativo de Cassandra v5 sobre TCP en el puerto 9042. Para el usuario de la terminal la consulta sigue un patrón de solicitud y respuesta. Entre nodos hay comunicación entre pares. El coordinador intercambia mensajes con las réplicas y el *gossip* circula periódicamente en segundo plano. Por eso el descubrimiento de cambios de estado y la espera de respuestas para una consulta son procesos distintos [4].

3.4. Tolerancia a fallos

Cada nodo estima la disponibilidad de sus pares a partir de la información de actividad que recibe. Si una réplica deja de responder, la petición puede seguir cuando hay suficientes copias alcanzables. En escrituras, el coordinador puede conservar una indicación temporal de la mutación pendiente para reenviarla cuando la réplica regrese. La reparación de lectura ayuda a reconciliar versiones según la configuración de la tabla y la reparación antientropía compara rangos entre réplicas para corregir diferencias [4]. Mi experimento examinó la continuidad de lectura de un registro ya replicado.

Discord documentó que los picos de solicitudes sobre determinadas particiones de mensajes saturaban algunos nodos y elevaban la latencia de otras peticiones. La réplica adicional protege frente a la pérdida de una copia, pero las particiones calientes también exigen controlar la carga. El equipo introdujo un servicio que agrupaba solicitudes idénticas y migró el almacenamiento de mensajes a ScyllaDB [1]. El caso amplía la discusión del laboratorio porque aborda degradación bajo carga, además de la pérdida de un nodo.

4. Experimentación propia

4.1. Objetivo y configuración

La prueba evaluó si Cassandra podía devolver el mismo registro con `ONE` y `QUORUM` al reducir el número de réplicas activas. El entorno usó Cassandra 5.0.9 en Docker, tres nodos en `dc1` y un factor de replicación de tres.

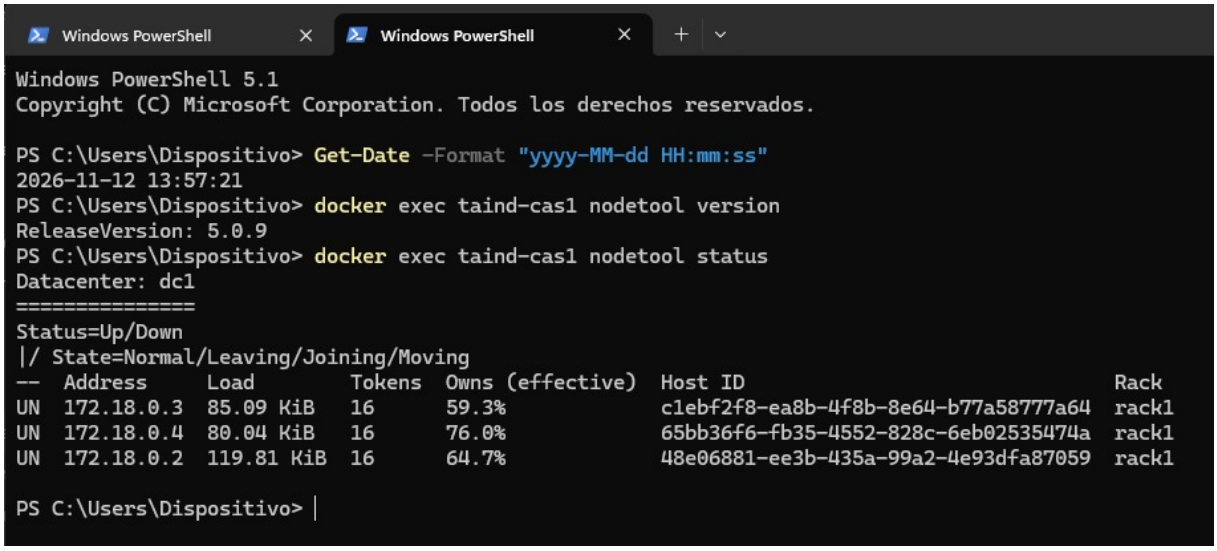


Figura 2: Estado inicial del clúster con tres nodos activos.

4.2. Procedimiento y resultados

El registro con `id = 1` se insertó con `ALL`, que exigía la confirmación de las tres réplicas. Se mantuvo activo `taind-cas1` como coordinador de la consulta. Los comandos y las salidas se documentan en `Experimento/prueba-cassandra.md`.

Con tres réplicas activas, ambas lecturas devolvieron el registro.

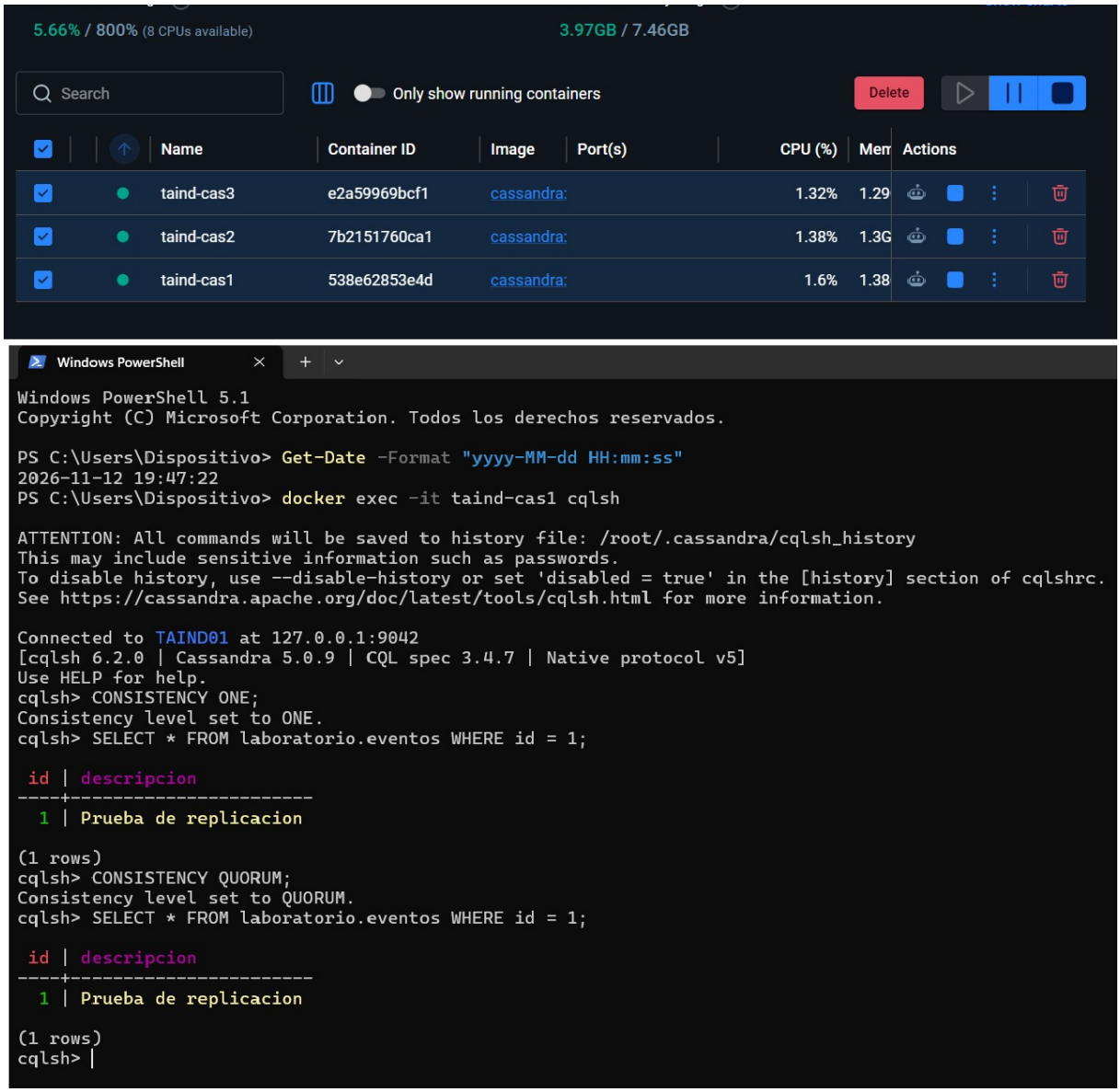


Figura 3: Lecturas con tres réplicas activas.

En la prueba con dos réplicas activas, Docker Desktop muestra que `taind-cas2` estaba detenido y que `taind-cas1` y `taind-cas3` seguían activos. En ese escenario, tanto `ONE` como `QUORUM` devolvieron el registro (Figura 4).

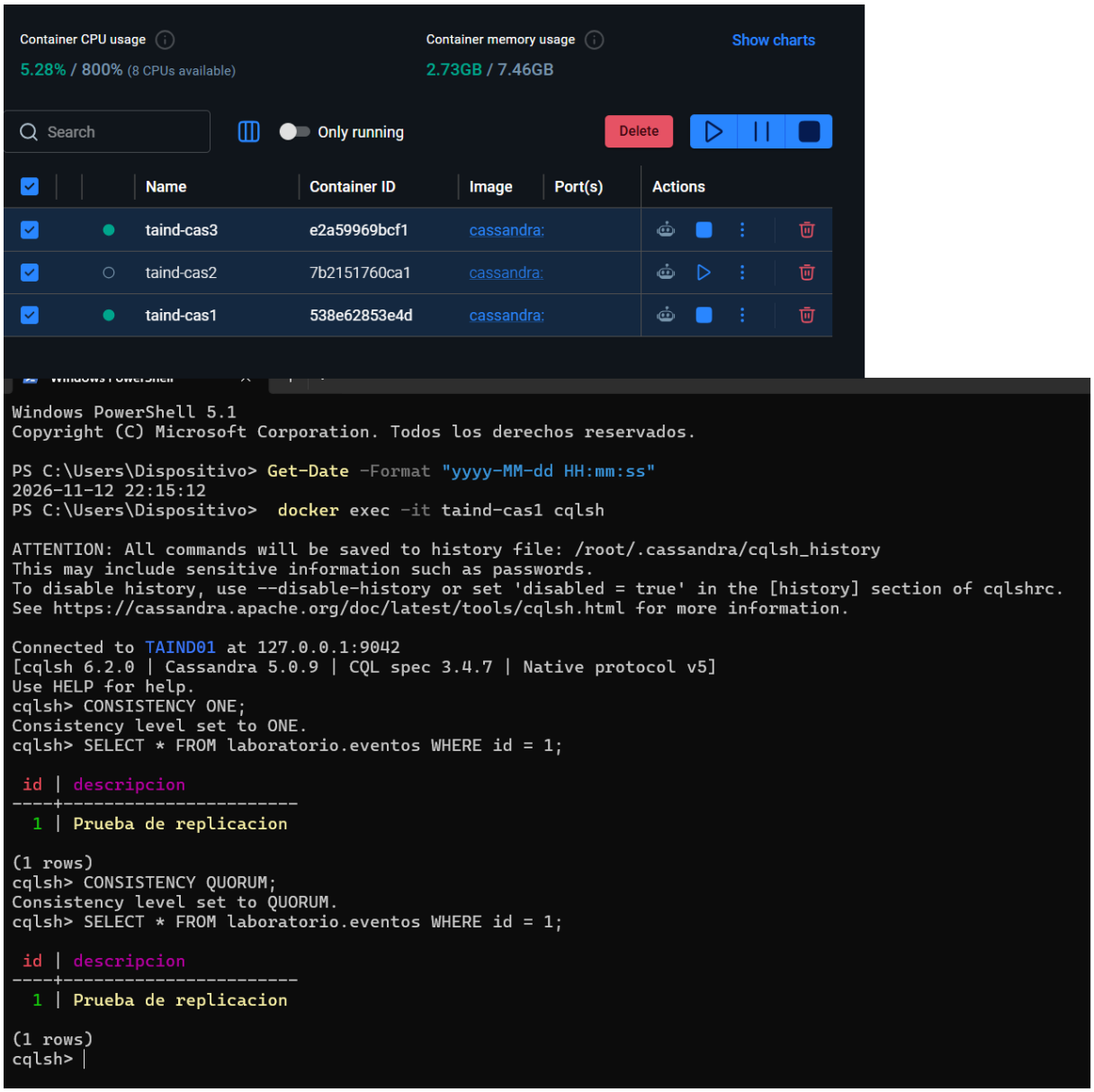


Figura 4: Lecturas con dos réplicas activas.

En la prueba con una sola réplica activa, `ONE` devolvió el registro y `QUORUM` produjo un error `Unavailable`. La salida indicó dos réplicas requeridas y una disponible.

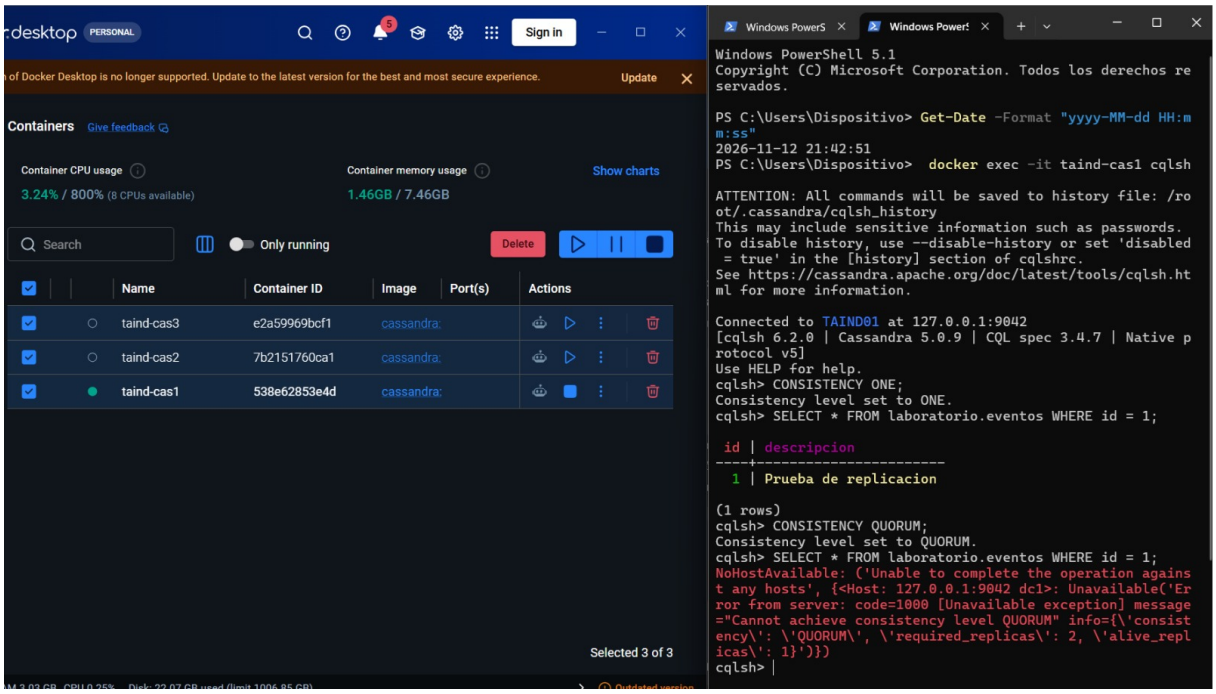


Figura 5: Resultado con una réplica activa.

4.3. Interpretación

Con el coordinador activo y el registro escrito previamente en las tres réplicas, la lectura pudo continuar con `ONE`. La misma operación dejó de estar disponible con `QUORUM` al faltar una respuesta para alcanzar la mayoría. El error coincide con el umbral que documenta Cassandra [4]. La prueba permite comparar la disponibilidad de la lectura ante la pérdida de nodos en esta configuración.

5. Ejemplos del entorno del estudiante

5.1. Entrega de una actividad en el SGA de la UTEQ

Cuando intenté subir una actividad, el SGA se quedó cargando. Al terminar la espera, la pantalla quedó como si yo no hubiera enviado nada. Volví a intentar la carga varias veces y seguí sin ver una confirmación. Cerré el SGA y, al abrirlo de nuevo, la actividad ya figuraba como cargada.

Relaciono esta experiencia con el modelo cliente-servidor y la confirmación de una solicitud. La pantalla inicial me dejó sin saber si la entrega se había registrado. Al volver a consultar el estado desde una nueva sesión, pude ver la actividad. Desde mi lado quedó sin determinar cuál de los intentos se registró.

5.2. Transferencia en Banco Pichincha

Al hacer una transferencia en la aplicación de Banco Pichincha, recibí un mensaje de error. Lo intenté de nuevo y volvió a aparecer el error. En un intento posterior la aplicación mostró que la transferencia se había realizado. Al revisar el resultado, vi que se había registrado una sola vez.

Este caso se relaciona con los reintentos y la idempotencia. Una solicitud idempotente puede repetirse sin producir un efecto adicional. En una transferencia, esto importa porque un mensaje de error puede dejar al usuario sin saber si la operación se registró. En mi caso observé una sola transferencia y, desde la aplicación, no pude determinar cómo se procesó cada intento.

6. Análisis comparativo

Cassandra y CockroachDB distribuyen datos y conservan copias para responder ante fallos. La comparación se centra en la lectura de datos replicados. Cassandra se evaluó en Docker y CockroachDB se analiza a partir de su documentación [4], [5], [6].

Cuadro 2: Comparación de mecanismos de Cassandra y CockroachDB.

Criterio	Cassandra	CockroachDB
Distribución	La clave de partición determina el token y la ubicación de las réplicas.	El espacio de claves se divide en rangos replicados.
Replicación	Las operaciones ordinarias usan un nivel de consistencia seleccionado para cada operación.	Cada rango mantiene un registro replicado mediante Raft.
Coordinación de lectura	El nodo que recibe la consulta coordina las respuestas de las réplicas.	La lectura actual pasa normalmente por el titular del arrendamiento del rango.
Disponibilidad ante fallos	Con una réplica alcanzable, `ONE` puede completar la lectura. `QUORUM` requiere una mayoría.	El progreso de las escrituras de un rango requiere una mayoría de sus réplicas votantes. Si se pierde el titular, el rango necesita una elección o transferencia válida para recuperar el servicio de lecturas actuales.
Recuperación	Las indicaciones pendientes, la reparación de lectura y la reparación antientropía ayudan a reconciliar réplicas.	Las réplicas pueden ponerse al día mediante entradas del registro de Raft o mediante una instantánea.
Latencia de lectura	El nivel elegido determina cuántas respuestas debe esperar el coordinador.	Una réplica cercana puede servir lecturas históricas autorizadas por una marca de tiempo cerrada.

El cuórum de lectura de Cassandra es un umbral para una solicitud concreta. En CockroachDB la mayoría de Raft se utiliza para acordar las actualizaciones de un rango. Una lectura histórica en CockroachDB tiene una antigüedad explícita y un límite de seguridad definido por el rango. La lectura `ONE` de Cassandra depende de la versión que posea la réplica consultada. Por eso conviene distinguir cuántas respuestas se requieren de qué versión se promete al cliente [4], [5], [6].

7. Postura crítica

La consistencia ajustable es una fortaleza cuando la aplicación puede decidir qué operaciones toleran leer una copia atrasada. Para un historial de eventos consultado por clave,

una lectura disponible durante la pérdida de dos réplicas puede ser más valiosa que exigir la mayoría en todas las consultas. La prueba mostró esa ventaja para un registro que ya existía en las tres réplicas. Si el dato hubiera cambiado durante el fallo, haría falta otra prueba para determinar qué versión observaría el lector.

El diseño obliga a conocer los patrones de acceso. Una aplicación que concentra peticiones sobre pocas claves puede saturar las réplicas responsables, como documentó Discord [1]. En bases NoSQL distribuidas entre regiones, los niveles de consistencia pueden afectar el tiempo de respuesta y el rendimiento de las operaciones [2], [7]. Para un sistema que deba comprobar en una misma transacción invariantes que abarcan múltiples registros, como evitar un cargo duplicado mientras se actualizan varios saldos, evaluaría primero una base SQL distribuida con garantías transaccionales adecuadas.

8. Conclusiones

Con tres y con dos réplicas activas, `ONE` y `QUORUM` devolvieron el mismo registro. Con una réplica activa, `ONE` conservó la disponibilidad de lectura. `QUORUM` devolvió un error que identificó el umbral requerido y las réplicas alcanzables. Por tanto, la cantidad de nodos que esta lectura puede perder depende del nivel elegido.

La comparación con CockroachDB muestra que contar réplicas es insuficiente para describir la garantía de un sistema. También importa si la mayoría se usa para responder una lectura, para replicar un cambio acordado o para sostener la coordinación tras un fallo. Para extender el experimento sería necesario introducir escrituras durante la caída y observar las versiones leídas y la convergencia después de la recuperación.

9. Declaración de uso de inteligencia artificial

Utilicé ChatGPT en modo Codex, con el modelo GPT-6.1 Sol, como apoyo para organizar el informe, mejorar la redacción de algunas secciones y revisar que las citas del texto correspondieran a las referencias. Durante la práctica, también lo consulté para entender los comandos de Docker y Cassandra y aclarar los mensajes de error de la terminal.

Elaboré el informe a partir de la práctica que realicé en mi equipo. Configuré el clúster de tres nodos, ejecuté las consultas con `ONE` y `QUORUM` y tomé las capturas de los resultados. Para revisar el documento, contrasté la interpretación de la prueba con mis capturas y cotejé los datos de los artículos citados con sus publicaciones originales.

Referencias

[1] B. Ingram. «How Discord Stores Trillions of Messages,» Discord. dirección: `https://discord.com/blog/how-discord-stores-trillions-of-messages`

[2] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, 2024, artículo 158. DOI: `10.1186/s13677-024-00716-7`

[3] J. Redžepagić, A. Kapulica, N. Malešević y V. Dakić, «Empirical Performance and Operational Analysis of Monolithic and Distributed Database Architectures in Kubernetes Environments,» *Computers*, vol. 15, n.º 5, 2026, artículo 282. DOI: `10.3390/computers15050282`

[4] Apache Software Foundation. «Dynamo,» Apache Cassandra 5.0 Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/dynamo.html

[5] Cockroach Labs. «Replication Layer,» CockroachDB Documentation, visitado 13 de nov. de 2026. dirección: https://www.cockroachlabs.com/docs/stable/architecture/replication-layer

[6] Cockroach Labs. «Follower Reads,» CockroachDB Documentation, visitado 13 de nov. de 2026. dirección: https://www.cockroachlabs.com/docs/stable/follower-reads

[7] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Benchmarking Consistency Levels of Cloud-Distributed NoSQL Databases Using YCSB,» *IEEE Access*, vol. 13, págs. 63 428-63 438, 2025. DOI: 10.1109/ACCESS.2025.3558923

Anexos

A. Esquema inicial a mano

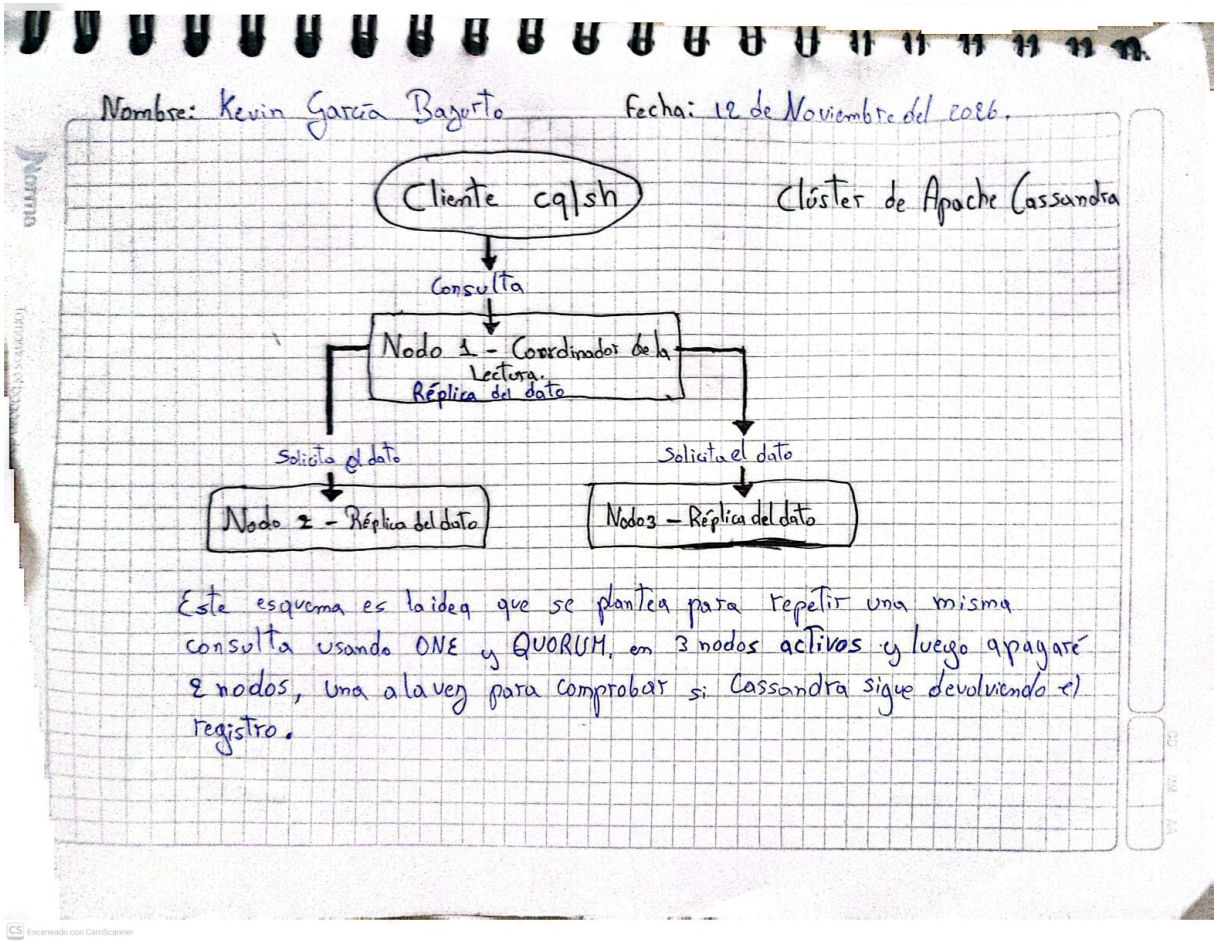


Figura 6: Bosquejo inicial del clúster Cassandra.

B. Evidencias de lectura de las fuentes

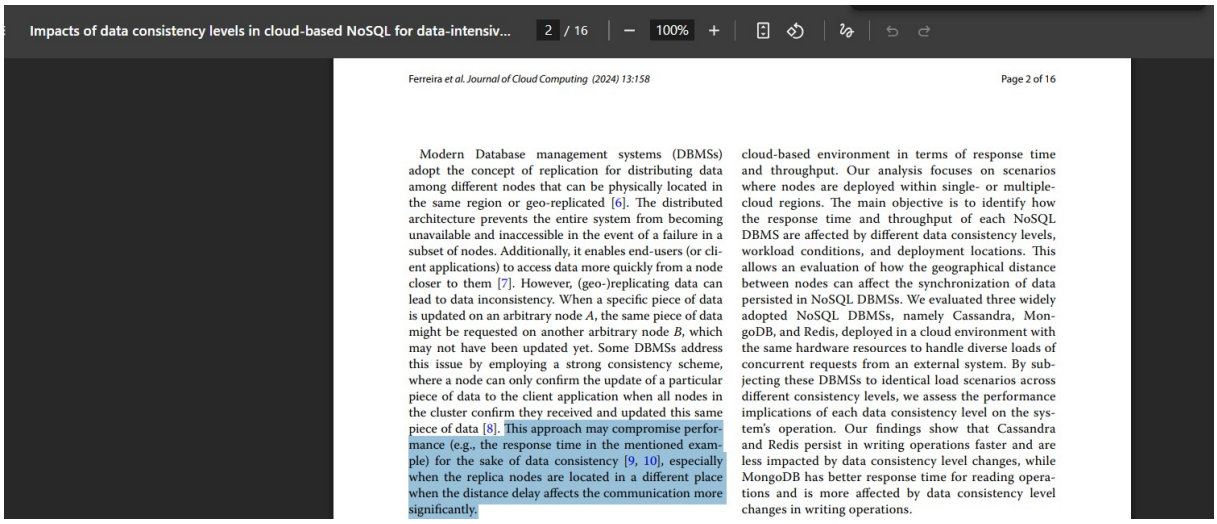


Figura 7: Ferreira et al., página 2. Efecto de la distancia entre réplicas en el tiempo de respuesta, citado en CAP y PACELC [2].

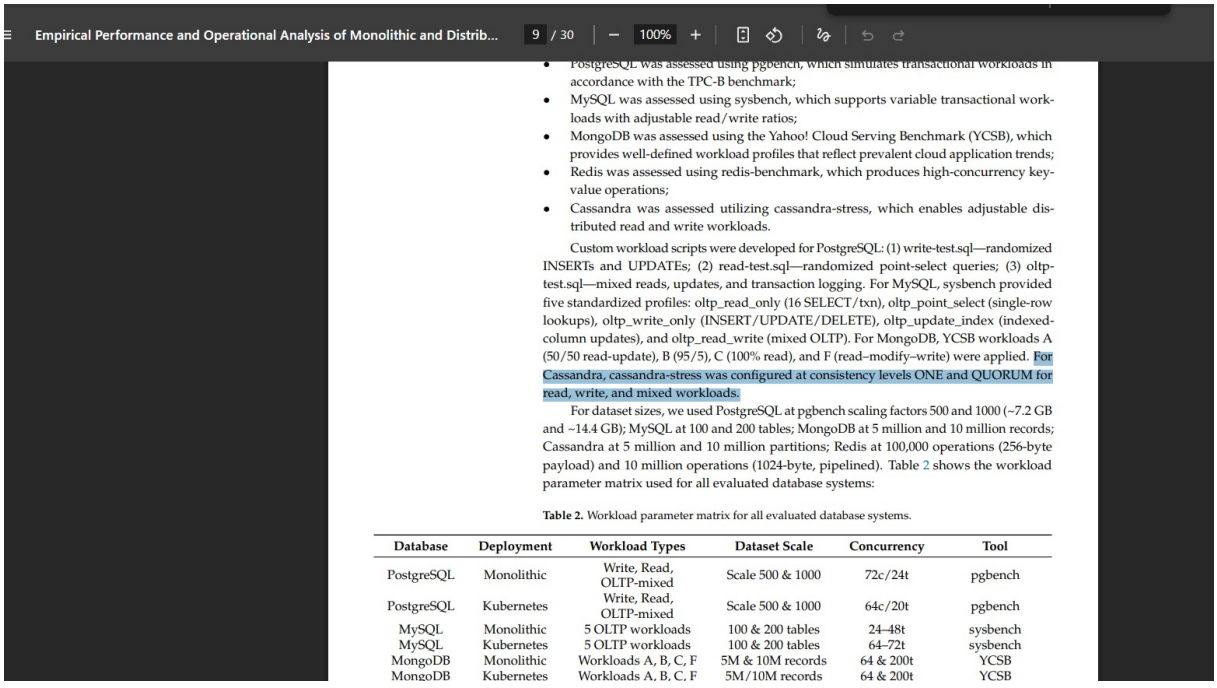


Figura 8: Redžepagić et al., página 9. Configuración de `ONE` y `QUORUM` en las pruebas de Cassandra, citada en la introducción y el análisis de consistencia [3].

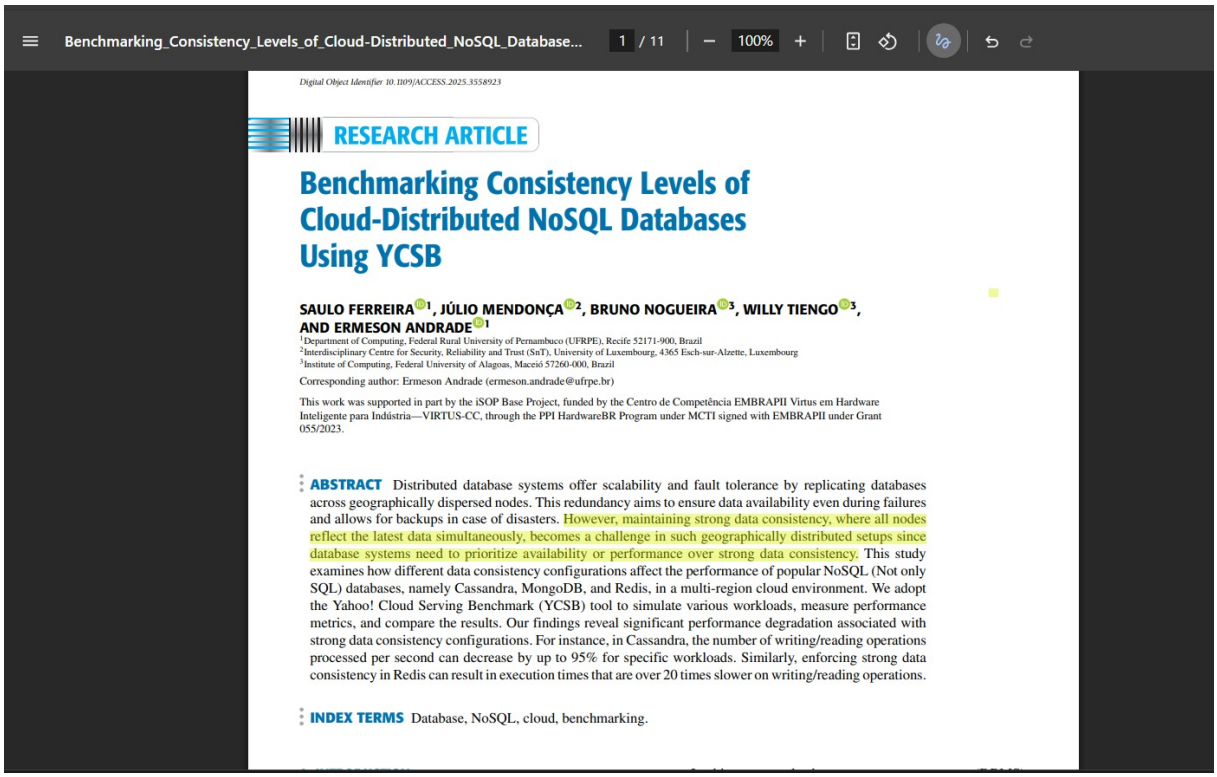


Figura 9: Ferreira et al., página 63428 (primera del PDF). Compromisos entre consistencia y rendimiento en despliegues multirregión, citados en la postura crítica [7].